

Operating Systems

501453-3

First Come First Serve (FCFS) CPU Scheduling Algorithm

Student Name	Student ID
Faris Althagafi (Leader)	-
Abdullah Saad Alqarni	-
Nawaf Ayidh Alqurashi	-
Turki Khulaif Almutari	-
Saad Saeed Almalki / 0xSaad	-
Saqer Adel Alotaibi	-

1. Problem Description

1.1 What is CPU Scheduling?

In a modern operating system, multiple processes compete for access to the CPU at the same time. Since a single-core CPU can only execute one process at a time, the operating system must decide which process to run next. This decision-making mechanism is called CPU scheduling.

CPU scheduling is one of the most fundamental concepts in operating systems. Without an effective scheduling strategy, some processes may never get to run, system resources may be wasted, and user experience can degrade significantly.

1.2 What is the FCFS Problem?

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm. In FCFS, processes are dispatched to the CPU in the exact order they arrive in the ready queue — just like a queue at a service counter.

While FCFS is easy to understand and implement, it introduces a well-known problem called the Convoy Effect:

- A long process that arrives early holds the CPU for an extended time.
- All shorter processes behind it must wait, even if they could finish in milliseconds.
- This causes unnecessarily high waiting times and poor overall system responsiveness.

1.3 Why is This Important in Operating Systems?

Understanding scheduling algorithms is critical because they directly impact:

- CPU utilization — keeping the processor as busy as possible.
- Throughput — the number of processes completed per unit of time.
- Turnaround time — the total time from process submission to completion.
- Waiting time — the total time a process spends waiting in the ready queue.
- Response time — the time from submission until the first response is produced.

2. Solution Explanation

2.1 Algorithm Overview

The FCFS algorithm operates on a simple, non-preemptive principle: once a process begins execution, it runs to completion before the CPU is given to the next process.

Step-by-Step Algorithm

1. Accept all process inputs: Process ID, Arrival Time (AT), and Burst Time (BT).
2. Sort processes in ascending order of Arrival Time.
3. Initialize the current time variable to 0.
4. For each process in the sorted order:
 - If the CPU is idle (current time < arrival time), advance current time to the arrival time.
 - Set Start Time (ST) = current time.
 - Set Completion Time (CT) = ST + BT.
 - Advance current time to CT.
5. Compute TAT and WT for each process.
6. Compute and display averages.

2.2 Mathematical Equations

The following equations are used to evaluate scheduling performance:

Metric	Formula	Description
Completion Time (CT)	$CT = ST + BT$	The time at which the process finishes execution
Turnaround Time (TAT)	$TAT = CT - AT$	Total time from arrival to completion
Waiting Time (WT)	$WT = TAT - BT$	Time spent waiting in the ready queue
Average TAT	$Avg\ TAT = \Sigma TAT / n$	Mean turnaround time across all processes
Average WT	$Avg\ WT = \Sigma WT / n$	Mean waiting time across all processes

2.3 Example Input

Consider four processes with the following arrival and burst times:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	2
P2	1	5
P3	2	4
P4	3	5

2.4 Step-by-Step Calculations

Process	AT	BT	ST	CT	TAT = CT - AT	WT = TAT - BT
P1	0	2	0	2	$2 - 0 = 2$	$2 - 2 = 0$
P2	1	5	2	7	$7 - 1 = 6$	$6 - 5 = 1$
P3	2	4	7	11	$11 - 2 = 9$	$9 - 4 = 5$
P4	3	5	11	16	$16 - 3 = 13$	$13 - 5 = 8$

Average Turnaround Time: $(2 + 6 + 9 + 13) / 4 = 30 / 4 = 7.5 \text{ ms}$

Average Waiting Time: $(0 + 1 + 5 + 8) / 4 = 14 / 4 = 3.5 \text{ ms}$

2.5 Gantt Chart

The Gantt chart below shows the order in which processes execute on the CPU over time:

P1		P2					P3				P4					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

2.6 Expected Behavior and Observations

- The algorithm is non-preemptive: once a process starts, it cannot be interrupted.
- Processes are served in the exact order of arrival — fair in the sense of arrival order.
- The Convoy Effect is visible: P3 and P4 wait significantly even though they have moderate burst times, because P2 (burst = 5) arrives before them.
- FCFS performs well when all burst times are similar, but poorly when they vary widely.

3. Example Input and Output

3.1 Program Input

The following input was provided to the FCFS scheduling program:

Process ID	Arrival Time	Burst Time
P1	0	2
P2	1	5
P3	2	4
P4	3	5

```
Enter number of processes: 4
Process ID: p1
Arrival Time: 0
Burst Time: 2
Process ID: p2
Arrival Time: 1
Burst Time: 5
Process ID: p3
Arrival Time: 2
Burst Time: 4
Process ID: p4
Arrival Time: 3
Burst Time: 5
```

3.2 Program Output

The program produces the following scheduling results:

```
=====
FCFS CPU Scheduling Results
=====
PID      AT      BT      CT      TAT     WT
-----
p1        0        2        2        2        0
p2        1        5        7        6        1
p3        2        4       11        9        5
p4        3        5       16       13        8
=====

Average Turnaround Time = 7.50
Average Waiting Time    = 3.50
Press any key to continue . . .
```

3.3 Explanation of Results

- P1 arrives at time 0 with the shortest burst time (2 ms). It starts immediately, finishes at time 2, and has zero waiting time — the best-case scenario.
- P2 arrives at time 1 but only waits 1 ms because P1 finishes at time 2. It then runs for 5 ms and completes at time 7.
- P3 arrives at time 2 but must wait until P2 finishes at time 7, resulting in a waiting time of 5 ms. This illustrates the Convoy Effect.
- P4 suffers the most: despite arriving early (time 3), it must wait for both P2 and P3 to complete, resulting in a waiting time of 8 ms.

This example demonstrates how FCFS can be unfair to shorter processes that arrive slightly later than a longer process.

4. Source Code (Python)

The following Python program implements the FCFS CPU scheduling algorithm. It accepts process details from the user, computes scheduling metrics, and displays a formatted results table.

```
#
# FCFS CPU Scheduling Algorithm
# Operating Systems Project
# Section: 2673
# <<< Dr. Alwaleed Alharbi >>>
#

def fcfs_scheduling(processes):
    """
    FCFS Scheduling Algorithm
    Input: list of processes (pid, at, bt)
    Output: processes with ct, tat, wt
    """

    # Sort processes by Arrival Time
    processes.sort(key=lambda p: p['at'])

    current_time = 0

    for process in processes:

        # If CPU is idle, jump to arrival time
        if current_time < process['at']:
            current_time = process['at']

        # Completion Time
        process['ct'] = current_time + process['bt']
        current_time = process['ct']

        # Turnaround Time
        process['tat'] = process['ct'] - process['at']

        # Waiting Time
        process['wt'] = process['tat'] - process['bt']

    return processes

def print_results(processes):
    """Display FCFS results in a table"""

    print("\n" + "=" * 50)
    print(" FCFS CPU Scheduling Results")
    print("=" * 50)

    print(f"{'PID':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
    print("-" * 50)

    for p in processes:
        print(f"{p['pid']:<10}{p['at']:<6}{p['bt']:<6}"
              f"{p['ct']:<6}{p['tat']:<6}{p['wt']:<6}")

    print("-" * 50)

    n = len(processes)

    avg_tat = sum(p['tat'] for p in processes) / n
    avg_wt = sum(p['wt'] for p in processes) / n

    print(f"\nAverage Turnaround Time = {avg_tat:.2f}")
    print(f"Average Waiting Time      = {avg_wt:.2f}")

def main():
    print("=== FCFS CPU Scheduling Simulator ===")
```

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    pid = input("Process ID: ")
    at = int(input("Arrival Time: "))
    bt = int(input("Burst Time: "))

    processes.append({
        'pid': pid,
        'at': at,
        'bt': bt
    })

result = fcfs_scheduling(processes)
print_results(result)

if __name__ == "__main__":
    main()
```


5. Discussion and Conclusion

5.1 Advantages of FCFS

- Simple and easy to implement.
- No starvation — every process eventually gets the CPU.
- Fair in terms of arrival order.
- Minimal overhead — no complex priority calculations needed.

5.2 Disadvantages of FCFS

- Convoy Effect: short processes wait behind long ones.
- High average waiting time when burst times vary widely.
- Not suitable for time-sharing or interactive systems.
- Non-preemptive: cannot respond to higher-priority urgent tasks.

5.3 Conclusion

The First-Come, First-Served (FCFS) scheduling algorithm is the most basic form of CPU scheduling. Its simplicity makes it an excellent teaching tool and a useful baseline for comparing other algorithms. However, real-world operating systems require more sophisticated approaches — such as Shortest Job First (SJF), Round Robin (RR), or Multilevel Queue Scheduling — to achieve lower average waiting times and better system responsiveness.

Through this project, we implemented and analyzed FCFS in Python, verified its mathematical formulas with a worked example, and visualized process execution using a Gantt chart. This hands-on experience has deepened our understanding of how operating systems manage CPU resources.

5.4 References

- Operating System Concepts — Abraham Silberschatz, Peter B. Galvin, Greg Gagne
- Modern Operating Systems — Andrew S. Tanenbaum
- Operating Systems: Internals and Design Principles — William Stallings
- GeeksforGeeks
- TutorialsPoint
- Javatpoint
- StudyTonight
- IBM Documentation
- MIT OpenCourseWare

— *End of Report* —
